

Аннотации типов и валидация данных

https://netology.ru/profile/program/bhembd-25-pdl-1/lessons/523600/lesson_items/2836908

Цель

Научиться пользоваться аннотацией типов и технологиями для валидации данных

Задача «Система управления библиотекой»

В этом задании вы создадите систему управления библиотекой, используя аннотации типов Python и Pydantic.

1. Базовые модели

Создайте следующие Pydantic-модели:

Book

- `title: str`
- `author: str`
- `year: int`
- `available: bool`

User

- `name: str`
- `email: str` (с валидацией `email`)
- `membership_id: str`

```
In [1]: from pydantic import BaseModel, EmailStr, field_validator
        from typing import List
```

```
In [2]: class Book(BaseModel):
        title: str
        author: str
        year: int
        available: bool = True
        categories: List[str] = []

        @field_validator("categories", mode="before")
        @classmethod
        def check_non_empty(cls, v: List[str]) -> List[str]:
            if not all(c.strip() for c in v):
```

```

        raise ValueError("Categories must not contain empty strings")
    return v

class User(BaseModel):
    name: str
    email: EmailStr
    membership_id: str

```

2. Функции с аннотациями типов

Напишите следующие функции, используя аннотации типов:

```

add_book(...) -> ...
find_book(...) -> ...
is_book_borrow(...) -> ...
return_book(...) -> ...

```

In [3]: `from typing import Optional`

In [4]:

```

def add_book(library_books: List[Book], book: Book) -> List[Book]:
    library_books.append(book)
    return library_books

def find_book(library_books: List[Book], title: str) -> Optional[Book]:
    for book in library_books:
        if book.title.lower() == title.lower():
            return book
    return None

def is_book_borrow(book: Book) -> bool:
    return not book.available

def return_book(book: Book) -> Book:
    book.available = True
    return book

```

3. Расширенная модель и валидация

Создайте модель `Library`:

- `books: ...`
- `users: ...`

Добавьте в модель `Book` поле `categories: List[str]` с валидацией.

Реализуйте метод `total_books() -> ...` для модели `Library`.

```
In [5]: class Library(BaseModel):
        books: List[Book]
        users: List[User]

        def total_books(self) -> int:
            return len(self.books)
```

4. Обработка ошибок и исключения

Создайте исключение `BookNotAvailable`.

Измените функцию `is_book_borrow`, чтобы она вызывала `BookNotAvailable` при необходимости.

Напишите декоратор `log_operation` для логирования операций с книгами*.

```
In [6]: import functools
```

```
In [7]: class BookNotAvailable(Exception):
        pass

        def log_operation(func):
            @functools.wraps(func)
            def wrapper(*args, **kwargs):
                result = None
                try:
                    result = func(*args, **kwargs)
                except Exception as e:
                    raise
                return result
            return wrapper

        @log_operation
        def borrow_book(library: Library, title: str, user: User) -> Book:
            book = find_book(library.books, title)
            if not book:
                raise ValueError("Book not found in library.")
            if not book.available:
                raise BookNotAvailable(f"Book '{title}' is already borrowed.")
            book.available = False
            return book
```

Проверка

```
In [8]: books = [
        Book(title="Clean Code", author="Robert C. Martin", year=2008, categories=["Pro
        Book(title="Clean Architecture", author="Robert C. Martin", year=2017, categori
        ]

        users = [
```

```
    User(name="Артём", email="func@example.com", membership_id="U001"),
    User(name="Сергей", email="reidj@example.com", membership_id="U002"),
]

library = Library(books=books, users=users)

assert library.total_books() == 2

book = find_book(library.books, "Clean Code")
assert book is not None
assert book.author == "Robert C. Martin"

borrow_book(library, "Clean Code", users[0])
assert not book.available

try:
    borrow_book(library, "Clean Code", users[1])
except BookNotAvailable:
    pass

return_book(book)
assert book.available
```